Hughes_CNET5050_Assignment_1_Code

by Ryan Hughes
Northeastern University
Master's in Complex Network Analysis

October 12, 2026

Introduction

In Part 1 of this assignment, you will analyze selected properties of random and real-world networks. Submit your written solutions as a PDF prepared using LaTeX or Microsoft Word.

You must also submit all code used to complete the assignment, either as separate files or as clearly formatted appendices in your write-up.

Ensure that your code is readable and appropriately commented. If you use generative AI tools such as ChatGPT, clearly identify the tasks for which they were used and include the relevant prompts and generated outputs.

In Part 2, you will develop your ability to critically read and present network science research papers.

Part 1

I. Random networks [25 marks]

(a) Consider the Erdos–Renyi random graph model G(n, p) with n = 1000 nodes and edge probability p = 0.01. Write down the probability distribution P(L = l) for the total number of edges L, and explain each term in your expression. Then calculate the expected number of edges, ⟨L⟩, for the specified values of n and p.

Probability Distribution:

$$P(L) = \binom{\frac{N(N-1)}{2}}{L} p^L (1-p)^{\frac{N(N-1)}{2} - L}$$

This equation reflects the probability distribution for some probability p , that there are exactly 'L = l' total number of edges in an Erdos–Renyi random graph model.

To explain further, there are 4 variables represented here: $P(L)$, the probability for L edges. L, the number of edges. N, the number of nodes. And, p, the probability of an edge existing between any pair of nodes.

These variables make up the following key terms of the equation:
p^L reflects the probability of drawing an edge an arbitrary amount of times equal to the value of L

$(1-p)^{\frac{N(N-1)}{2}-L}$ reflects the probability we don't draw an edge for all other pairs of nodes that one could place an edge.

Or, in other words: the probability we don't make an edge over the remaining "N choose two $(\frac{N(N-1)}{2})$" pairs leftover after L pairs have been processed. And,

$\binom{\frac{N(N-1)}{2}}{L}$ represents "N choose 2, choose L" or, the picking of 2 unique nodes, an arbitrary amount of times: L.

Calculating the expected number of edges for G(1000,0.01):

$$\langle L \rangle = p\frac{N(N-1)}{2}$$

$$\langle L \rangle = 0.01\frac{1000(1000-1)}{2}$$

$$\langle L \rangle = 0.01 * \frac{1000(999)}{2}$$

$$\langle L \rangle = 0.01 * 499500$$

$$\langle L \rangle = 4995$$

(b) Write down the degree distribution P(K = k) and explain each term in your expression. Calculate the expected degree, $\langle k \rangle$, for the specified values of n and p.

Degree Distribution:

$$p_k = \binom{(N-1)}{k} p^k (1-p)^{(N-1)-k}$$

This equation reflects the probability distribution for some probability p , that there are exactly 'K = k' total degree in an Erdos–Renyi random graph model.

To explain further, there are just 3 variables in this equation. p_k, the probability for k degree to occur for a given node. N, the number of nodes. And, p, the probability of an edge existing between any pair of nodes.

These variables make up the following key terms of the equation:
p^k reflects the probability of a node having a degree of an arbitrary amount of times, equal to the value of k

$(1-p)^{(N-1)-k}$ in a similar fashion, reflects the probability we don't have an edge (i.e., an edge contributing to the degree of a given node) for all other possibilities available.

$\binom{(N-1)}{k}$ represents "N-1 choose k" or, the picking of one other node, an arbitrary amount of times equal to the value of k.

Calculating the expected average degree for G(1000,0.01):

$$\langle k \rangle = p(N-1)$$

$$\langle k \rangle = 0.01(1000-1)$$

$$\langle k \rangle = 0.01 * (999)$$

$$\langle k \rangle = 9.99$$

(c) Write down the definition for average local clustering coefficient. What is the average local clustering coefficient of G(n, p), for the specified values of n and p?

$C_i = p$, is the definition for the average local clustering coefficient in Erdos-Renyi Random Graph models. Due to the fact, the placing of any edge is simply p, the odds of a node's neighbors being connected (and thereby, there being clustering) is also p.

As such, the average local clustering coefficient for $G(1000, 0.01)$ is:

$$C_i = p$$

$$C_i = 0.01$$

(d) Write down the approximate expression for the average shortest-path length $\langle d \rangle$ between nodes in G(n, p) and calculate it for the specified values of n and p.

$d \approx \frac{logN}{log\langle k \rangle}$, is the approximate expression for the average shortest-path length, $\langle d \rangle$, between nodes in a graph $G(N, p)$.

Therefore, for $G(N, p) = G(1000, 0.01)$:

$$d \approx \frac{logN}{log\langle k \rangle}$$

$$d \approx \frac{log1000}{log9.99}$$

$$d \approx \frac{3}{0.9995655}$$

$$d \approx 3.001304$$

(e) Write a program in Python, or another programming language of your choice, that takes n and p as inputs and returns one realization of G(n,p).

[16]:
```
from itertools import combinations
import networkx as nx
import numpy as np
import scipy.stats


def create_erdos_renyi_random_graph(N, p):
    """
    Creates a Erdos-Renyi Random Graph with 'N' nodes and an edge probability,␣
 ↪'p'

    Parameters
    ----------
    N : int
        The number of nodes of the graph to be created.

    p : float
        The probability an edge is placed between any pair of nodes

    Returns
    -------
    G : networkx.Graph
        The resulting erdos-renyi random graph,undirected, as supported by␣
 ↪NetworkX.

    """
    #Initialize the Graph
    from itertools import combinations
    import networkx as nx
    G = nx.Graph()
    #Initialize chance
    chance = 0

    #Add in all the nodes
    for i in range(N):
        #Add a node
        G.add_node(i)
    #Add in all the edges
```

```
    for node1_id,node2_id in combinations(G.nodes(), 2):#For every pair of nodes␣
 ↪within G
        chance = np.random.rand()#Draw a random chance 0-100%
        if chance <= p:#If the chance is within the threshold 'p'
            #Add an edge between that pair of nodes
            G.add_edge(node1_id,node2_id)
    return G

N = 100
p = 0.01
print(nx.erdos_renyi_graph(100,0.01))
print(create_erdos_renyi_random_graph(N,p))
```

```
Graph with 100 nodes and 57 edges
Graph with 100 nodes and 62 edges
```

(i) Generate NG = 100 independent graph realizations. For each realization, record the total number of edges. Plot the empirical distribution of these 100 edge counts and overlay the theoretical distribution derived in part (a). Clearly label the axes and legend.

[17]:
```
import networkx as nx
import matplotlib.pyplot as plt
from scipy.stats import binom
from scipy.special import comb

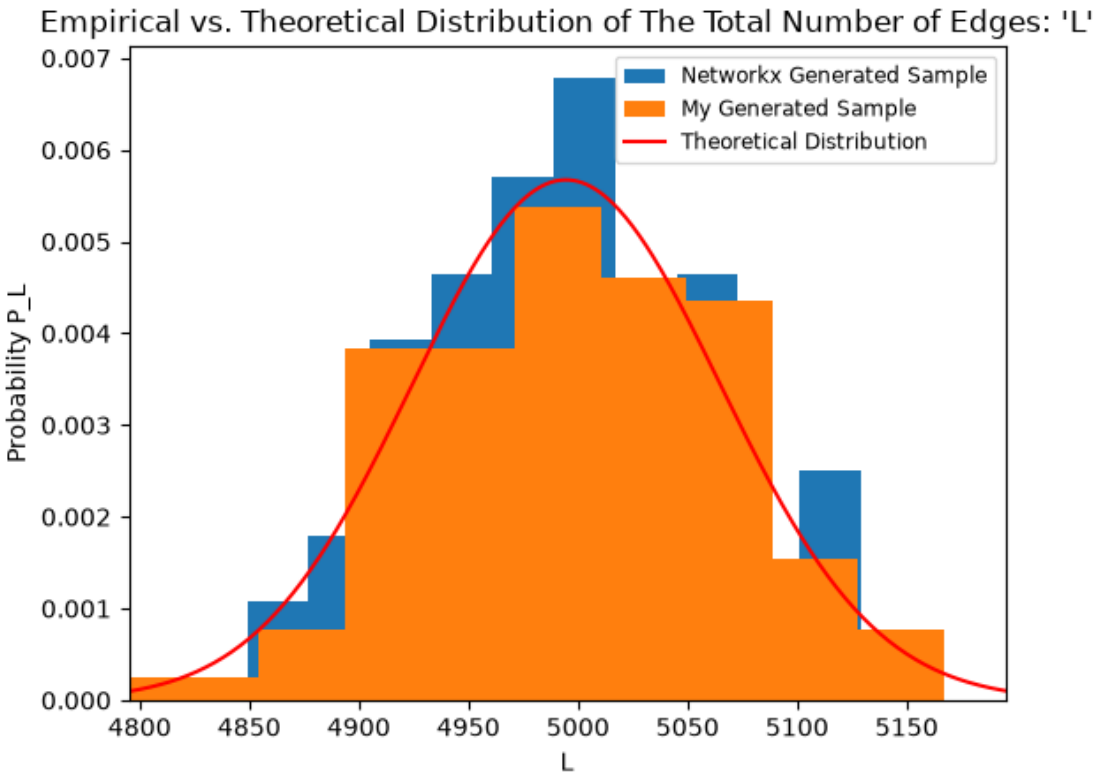
N=1000
p=0.01
num_edges_list=[]
my_num_edges_list=[]
for i in range(100):
  G=nx.erdos_renyi_graph(N, p)
  edges = list(G.copy().edges())
  num_edges_list.append(len(edges))
  G2=create_erdos_renyi_random_graph(N, p)
  edges = list(G2.copy().edges())
  my_num_edges_list.append(len(edges))

L = np.arange(4995-200,4995+200+1,step=1,dtype = (int)) #Picking a handful of␣
 ↪values centered around the average I calculated earlier of: 4995


part = (N * (N - 1)) // 2 #part = little 'n' in nCr
#P_l = comb(part, L)*(p**L)*(1-p)**(part-L)
P_l = binom.pmf(L, part, p)#Finding P_l with the binom() function from scipy␣
 ↪since comb() was throwing errors due to too large L

#Starting the plotting process
```

```
fig, ax = plt.subplots()#Note: fig is the plot container, while ax (i.e., the␣
 ↪'axes') is the plot itself
ax.hist(num_edges_list,label="Networkx Generated Sample",density=True)#Plotting␣
 ↪the networkx based random graph empirical data
ax.hist(my_num_edges_list,label="My Generated Sample",density=True)#Plotting my␣
 ↪function based random graph empirical data
ax.plot(L,P_l, color = 'r',label="Theoretical Distribution") #Plotting the␣
 ↪Theoretical
#Specifying figure details below:
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles=handles, labels=labels,loc="upper right",fontsize="small")
ax.set_title("Empirical vs. Theoretical Distribution of The Total Number of␣
 ↪Edges: 'L'")
ax.set_xlabel("L")
ax.set_ylabel("Probability P_L")
ax.set_xlim(np.min(L),np.max(L))
plt.show()
```



(ii) Generate NG = 100 independent graph realizations. For every realization, record the degree of each node and concatenate the results into a single sample containing 100 × n degree observations. Plot the empirical degree distribution and overlay the

theoretical distribution derived in part (b). Clearly label the axes and legend.

[18]:
```
import networkx as nx
import matplotlib.pyplot as plt
from scipy.stats import binom
from scipy.special import comb

N=1000
p=0.01
num_degree_list=[]
my_num_degree_list=[]
for i in range(100):
  G=nx.erdos_renyi_graph(N, p)
  degree = dict(G.copy().degree())
  deg= list(degree.values())
  num_degree_list.append(deg)
  G2=create_erdos_renyi_random_graph(N, p)
  degree = dict(G2.copy().degree())
  deg= list(degree.values())
  my_num_degree_list.append(deg)

k = np.arange(9.99-9.99,9.99+9.99+0.01,step=0.01,dtype = float)#Picking a␣
 ↪handful of values centered around the average I calculated earlier of: 9.99

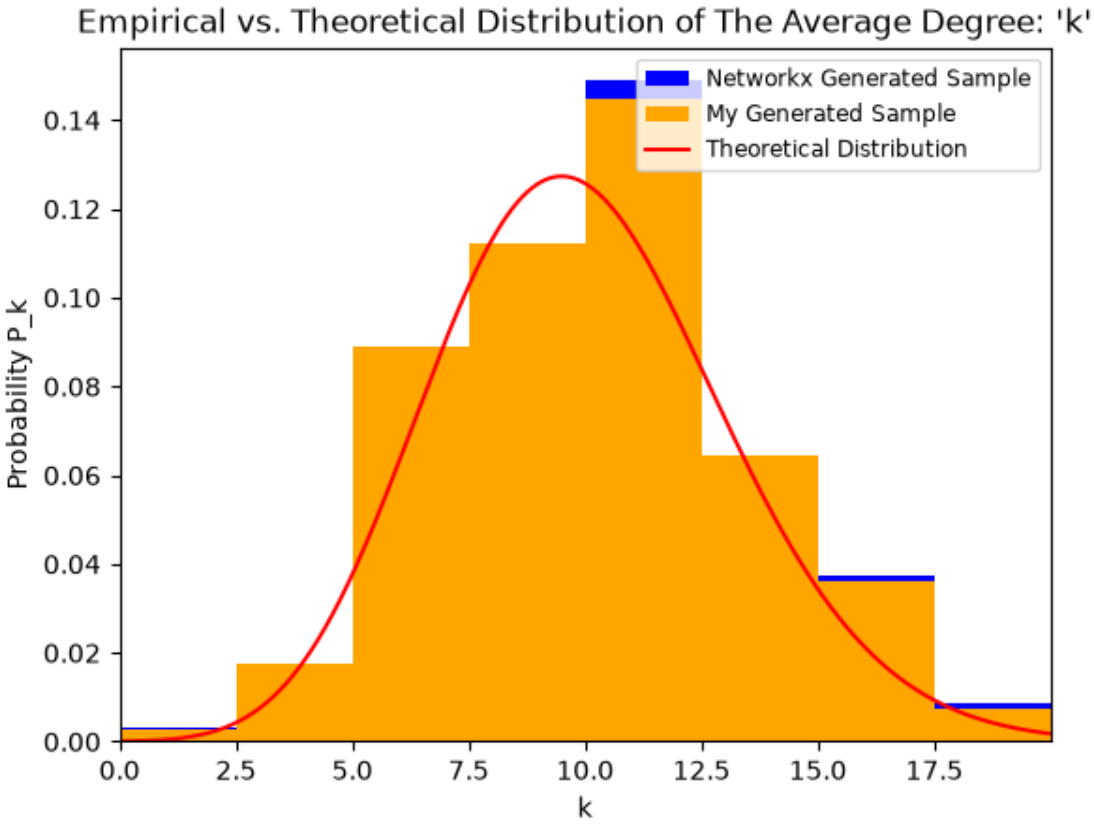
part = (N - 1) #part = little 'n' in nCr
P_l = comb(part, k)*(p**k)*(1-p)**(part-k)#Calculating P_l with comb()
#P_l = binom.pmf(k, part, p) #Unfortunately because nCr requires integers as␣
 ↪input, I can only calculate P_l for integer numbers with binom()
print("P_l: ",P_l)
print("k: ",k)

#Starting the plotting process
fig, ax = plt.subplots() #Note: fig is the plot container, while ax (i.e., the␣
 ↪'axes') is the plot itself
ax.hist(num_degree_list,label="Networkx Generated Sample",density=True,color =␣
 ↪'b',histtype='stepfilled')#Plotting Networkx based empirical
ax.hist(my_num_degree_list,label="My Generated Sample",density=True,color =␣
 ↪'orange',histtype='stepfilled')#Plotting My Random Graph based empirical
ax.plot(k,P_l, color = 'r',label="Theoretical Distribution")   #Plotting the␣
 ↪Theoretical

#Specifying figure details below:
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles=handles, labels=labels,loc="upper right",fontsize="small")
ax.set_title("Empirical vs. Theoretical Distribution of The Average Degree: 'k'")
ax.set_xlabel("k")
ax.set_ylabel("Probability P_k")
ax.set_xlim(np.min(k),np.max(k))
```

```
plt.show()
```

```
P_l:  [4.36073206e-05 4.48819923e-05 4.61864302e-05 ... 1.81293599e-03
 1.79979733e-03 1.78674498e-03]
k:  [0.000e+00 1.000e-02 2.000e-02 ... 1.997e+01 1.998e+01 1.999e+01]
```



II. Real-world networks [25 marks]

Download the airport-network.gml dataset from https://tinyurl.com/4rfrx4p9. Using the NetworkX library in Python, load the dataset with nx.read_gml and complete the following analyses.

Report the number of nodes n, the number of edges L, the average degree ⟨k⟩, and the average local clustering coefficient of the network.

[19]:
```
#Reading in the Data
Airport = nx.read_gml('C:/Users/rh614/OneDrive/Documents/ALL REPOS/Python/
 ↪Fundamentals-of-Complex-Networks/Assignment_1/data/airport-network.gml')
N = Airport.number_of_nodes()
print("N: ",N)
L = Airport.number_of_edges()
print("L: ",L)
avg_deg= np.mean(list(dict(Airport.degree()).values()))
```

```
print("Average Degree: ",avg_deg)
C_i = nx.average_clustering(Airport)
print("C_i: ",C_i)
```

```
N:  1574
L:  17215
Average Degree:  21.874205844980942
C_i:  0.504152539209577
```

Plot the empirical degree distribution using logarithmic scales on both axes. Bin the node degrees and plot a normalized histogram. Clearly label the axes and briefly comment on the shape of the distribution.

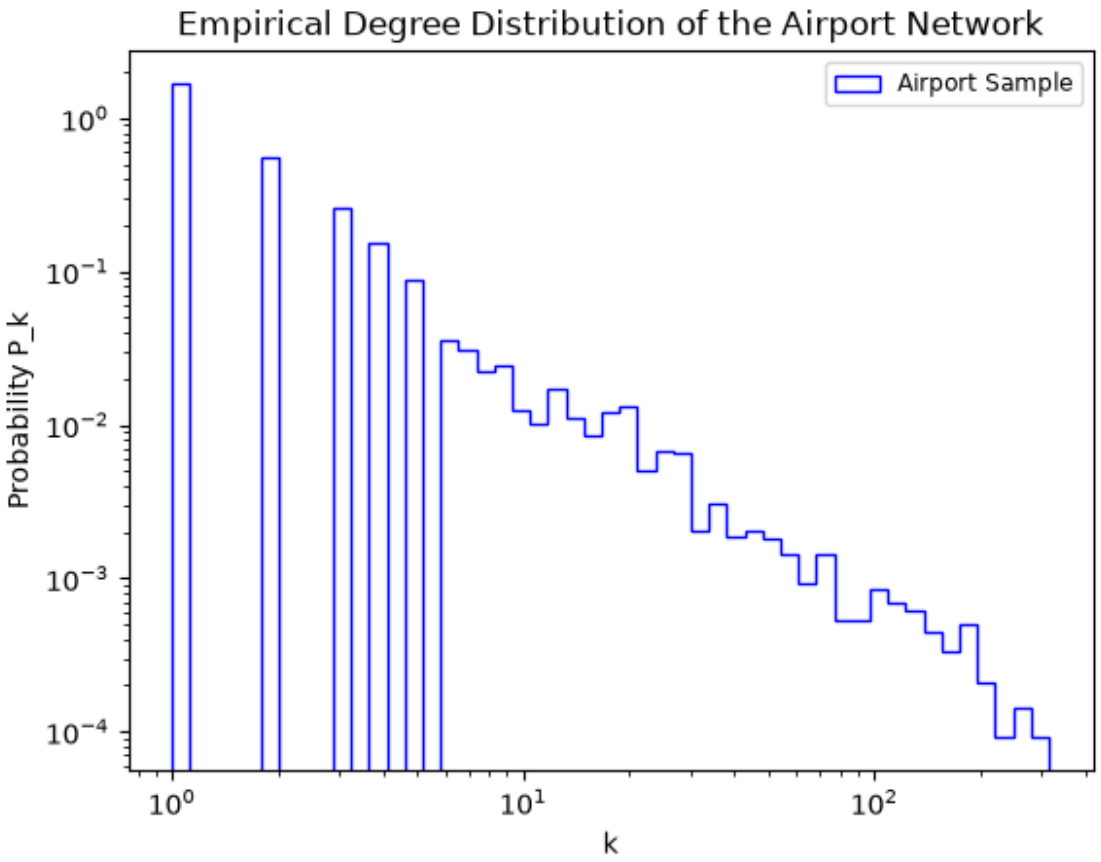
[20]:
```
deg = list(dict(Airport.degree()).values()) #Gathering the degree values as a␣
 ↪list
#Printing the Min and Max Degree for reference purposes
print("Max Deg: ",np.max(deg))
print("Min Deg: ",np.min(deg))

#Starting the plotting process
fig, ax = plt.subplots()
bins = np.geomspace(np.min(deg),np.max(deg)) #Making bins that are log-scaled

#Plotting the degree distribution histogram using log-scaled bins
ax.hist(deg,bins=bins,label="Airport Sample",density=True,color =␣
 ↪'b',histtype='step',log=True)
ax.loglog()#Making the axes log-log

#Specifying figure details below:
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles=handles, labels=labels,loc="upper right",fontsize="small")
ax.set_title("Empirical Degree Distribution of the Airport Network")
ax.set_xlabel("L")
ax.set_ylabel("Probability P_L")
plt.show()
```

```
Max Deg:  314
Min Deg:  1
```



The shape of the distribution is approximately a straight line with negative slope on the log-log scale. This indicates the data follows a power-law or truncated power-law distribution. The range of the degree distribution varies from 1 to 314, with nodes having a higher probability of having a degree that is lower (e.g, less than 10). 1 being the most probable degree to occur for a given node, and 314 the least probable to occur.

Overlay a Poisson degree distribution with the same mean degree ⟨k⟩ on the empirical degree distribution. Compare the two distributions and briefly discuss their main differences.

[21]:
```
from scipy.stats import poisson
deg = list(dict(Airport.degree()).values())
#Printing the Min and Max Degree for reference purposes
print("Max Deg: ",np.max(deg))
print("Min Deg: ",np.min(deg))
avg_deg= np.mean(list(dict(Airport.degree()).values()))#Getting the average␣
↪degree for the Poisson

#Starting the plotting process
fig, ax = plt.subplots()
```

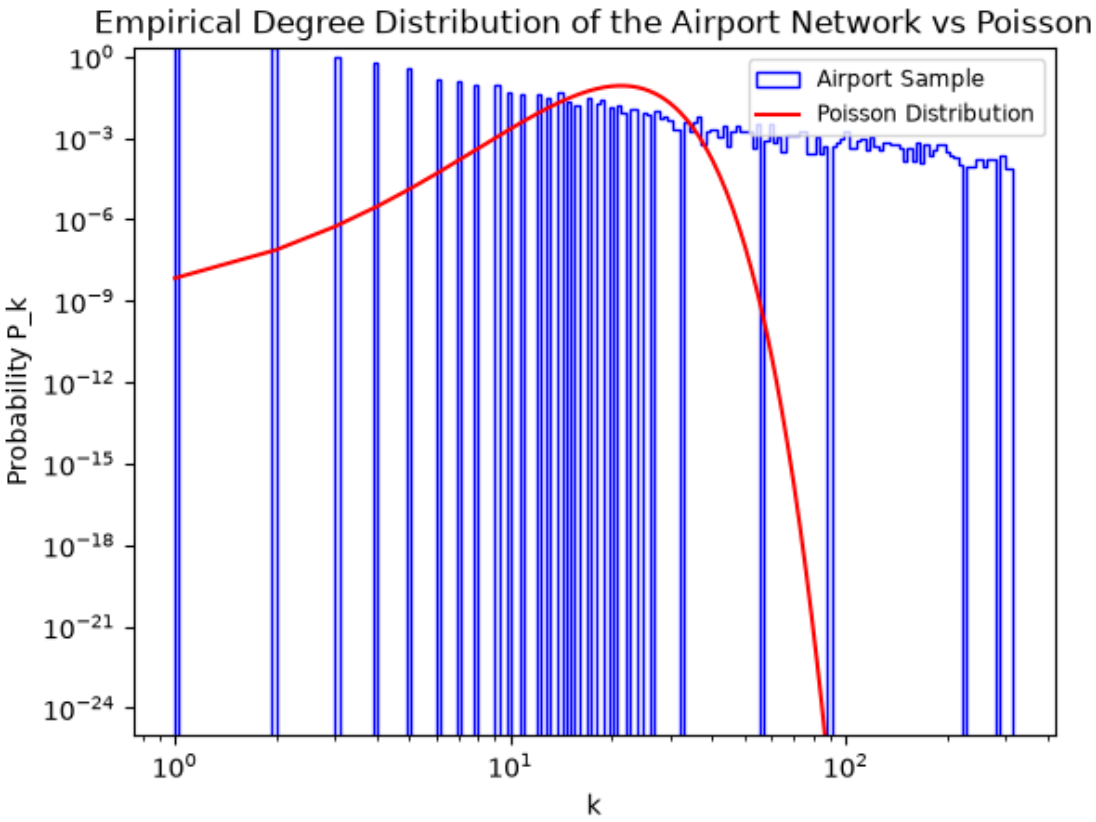
```
bins = np.geomspace(np.min(deg),np.max(deg),num=200)#Making log scaled bins for␣
 ↪the empirical data

#Note mu=lambda, k=x
k_values = range(np.min(deg),np.max(deg)+1,1)#Generating the input to the␣
 ↪poisson PDF as all integers of k from empirical-data
poisson = poisson.pmf(k=k_values,mu=avg_deg)#Getting the P_k based on Poisson

#Plotting the Empirical and Poisson Distributions
ax.hist(deg,bins=bins,label="Airport Sample",density=True,color =␣
 ↪'b',histtype='step',log=True)#Plotting the Empirical
ax.loglog()#Setting to log-log scale
ax.plot(k_values,poisson,label="Poisson Distribution",color = 'r')#Plotting the␣
 ↪Poisson

#Specifying figure details below:
handles, labels = ax.get_legend_handles_labels()#Gathering the labels to make␣
 ↪legends per plot
ax.legend(handles=handles, labels=labels,loc="upper␣
 ↪right",fontsize="small")#Adding legend
ax.set_title("Empirical Degree Distribution of the Airport Network vs Poisson")
ax.set_xlabel("k")
ax.set_ylim(10e-26,2)
ax.set_ylabel("Probability P_k")
plt.show()
```

```
Max Deg:  314
Min Deg:  1
```



Notably, the main difference is that the airport samples distribution follows closer to a straight line (i.e., a power law distribution, or truncated power law distribution) while the Poisson Distribution on the log-log scale doesn't represent the data well. The Poisson sharply cuts down in probability before the degree reaches 100, while the airport clearly still has many nodes (*relatively speaking*) with degree higher than 100.

Acknowledgements:
AI was not used in the creation of this assignment. I utilized stack-overflow for guidance, especially on how to generate the Binomial distribution and Poisson distribution respectively. I also referred to our coding lectures in CNET5051 and Python documentation pages for guidance on specific packages (e.g., Numpy, NetworkX, and SciPy) to understand how to leverage relevant commands correctly, resolve bugs and fix errors in my code.